DT-1 — DATA MODELING (FINAL)

Document Processing Analytics Platform

Focus: Load Reporting + Pipeline Health + Invoice Extraction | Multi-tenant, Role-based | Queries respond in 3–5 seconds

This document is the FINAL, locked data model for DT-1. It answers all 5 required data-modeling questions plus the remediation feature and confidence logic. Use it as the basis for DT-2 (API Design).

Overview — 11 Tables across 4 Layers

```
LAYER 1 · IDENTITY     → Tenants · Sites · Users · UserSiteAccess
LAYER 2 · PIPELINE     → Transactions · Files · FileStepHistory
LAYER 3 · SUPPORT      →  ErrorCatalog · ActivityLog
LAYER 4 · EXTRACTION   →  DocumentTypes · ExtractedFields
```

Layer 1 — Identity (who & access)

1. TENANTS — customer companies

Column	Type	Constraint
id	UUID	PK
name	VARCHAR(100)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL
is_active	BOOLEAN	NOT NULL

2. SITES — physical locations of a tenant

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
name	VARCHAR(100)	NOT NULL
location	VARCHAR(200)	NULLABLE
created_at	TIMESTAMPTZ	NOT NULL
is_active	BOOLEAN	NOT NULL

3. USERS — dashboard logins

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
email	VARCHAR(200)	UNIQUE
password_hash	VARCHAR(500)	NOT NULL
role	VARCHAR(50)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL
is_active	BOOLEAN	NOT NULL

4. USER_SITE_ACCESS — bridge (which user sees which site)

Column	Type	Constraint
id	UUID	PK
user_id	UUID	FK → Users
site_id	UUID	FK → Sites
granted_at	TIMESTAMPTZ	NOT NULL
(unique)		UNIQUE(user_id, site_id)

Layer 2 — Pipeline (the core)

5. TRANSACTIONS — the "TId" rows on the Load Reporting dashboard

Column	Type	Constraint
id	UUID	PK (the TId)
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
state	VARCHAR(20)	NOT NULL
source_system	VARCHAR(100)	NOT NULL
total_files	INTEGER	NOT NULL
uploaded_count	INTEGER	NOT NULL
processing_count	INTEGER	NOT NULL
failed_count	INTEGER	NOT NULL
completed_count	INTEGER	NOT NULL
submitted_at	TIMESTAMPTZ	NOT NULL
last_updated_at	TIMESTAMPTZ	NOT NULL
completed_at	TIMESTAMPTZ	NULLABLE

The 4 counters are pre-aggregated so the dashboard loads instantly (no counting 1M rows live).

6. FILES — generic file records (~1M rows)

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
transaction_id	UUID	FK → Transactions
document_type_id	UUID	FK → DocumentTypes (NULLABLE)
file_name	VARCHAR(255)	NOT NULL
file_type	VARCHAR(50)	NOT NULL
status	VARCHAR(20)	NOT NULL
current_step	VARCHAR(50)	NOT NULL
file_size_bytes	BIGINT	NULLABLE
extraction_status	VARCHAR(20)	NULLABLE
extraction_confidence	DECIMAL(4,3)	NULLABLE
last_updated_at	TIMESTAMPTZ	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL

7. FILE_STEP_HISTORY — step-by-step audit per file

Column	Type	Constraint	Note
id	UUID	PK	

file_id	UUID	FK → Files	
document_type_id	UUID	FK → DocumentTypes (NULLABLE)	NEW – denormalized for fast error panel
step_name	VARCHAR(50)	NOT NULL	Upload/Validate/Transform/Load
status	VARCHAR(20)	NOT NULL	
started_at	TIMESTAMPTZ	NULLABLE	
completed_at	TIMESTAMPTZ	NULLABLE	
error_code	VARCHAR(50)	NULLABLE	links to ErrorCatalog
error_message	TEXT	NULLABLE	

Decision: one row per step (not a JSON array) → so steps are queryable, sortable, groupable & indexable.

Layer 3 — Support

8. ERROR_CATALOG — known errors + how to fix them

Column	Type	Constraint
id	UUID	PK
error_code	VARCHAR(50)	UNIQUE
description	TEXT	NOT NULL
remediation_msg	TEXT	NULLABLE
created_at	TIMESTAMPTZ	NOT NULL
updated_at	TIMESTAMPTZ	NOT NULL

Powers the remediation popup — turns ERR_BAD_SCHEMA into "Check your column headers."

9. ACTIVITY_LOG — append-only system audit trail

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
event_type	VARCHAR(100)	NOT NULL
entity_type	VARCHAR(50)	NOT NULL
entity_id	UUID	NOT NULL
entity_name	VARCHAR(255)	NULLABLE
old_state	VARCHAR(50)	NULLABLE
new_state	VARCHAR(50)	NULLABLE
triggered_by	VARCHAR(100)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL

Layer 4 — Extraction

10. DOCUMENT_TYPES — catalog of supported types (extensible)

Column	Type	Constraint
id	UUID	PK
type_name	VARCHAR(100)	UNIQUE
category	VARCHAR(10)	NOT NULL (PDF/CSV)
description	TEXT	NULLABLE
is_active	BOOLEAN	NOT NULL

created_at	TIMESTAMPTZ	NOT NULL

11. EXTRACTED_FIELDS — flexible key-value extraction (EAV pattern)

Column	Type	Constraint
id	UUID	PK
file_id	UUID	FK → Files
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
field_name	VARCHAR(100)	NOT NULL
field_value	TEXT	NULLABLE
confidence	DECIMAL(4,3)	NULLABLE
is_valid	BOOLEAN	NOT NULL
extracted_at	TIMESTAMPTZ	NOT NULL

EAV = any new doc type stores its fields as rows here → ZERO schema migrations ever.

Relationships

From → To	Type
Tenants → Sites	1:N
Tenants → Users	1:N
Sites → Transactions	1:N
Transactions → Files	1:N
Files → FileStepHistory	1:N
Files → ExtractedFields	1:N
DocumentTypes → Files	1:N
DocumentTypes → FileStepHistory	1:N
Users ↔ Sites (via UserSiteAccess)	N:M
FileStepHistory → ErrorCatalog (by error_code)	soft ref

Indexes (why each exists)

Table	Index	Purpose
TRANSACTIONS	(tenant_id, site_id, last_updated_at DESC)	Main load-reporting view
TRANSACTIONS	(tenant_id, site_id, state)	Filter by state
FILES	(transaction_id)	Drill-down: files in a transaction
FILES	(tenant_id, site_id, status, last_updated_at DESC)	Recent failures
FILES	(tenant_id, site_id, document_type_id)	"Show all invoices"
FILE_STEP_HISTORY	(file_id)	File step drill-down + log download
FILE_STEP_HISTORY	(step_name, status)	Error analysis by step
EXTRACTED_FIELDS	(file_id)	Load a file's fields
EXTRACTED_FIELDS	(tenant_id, site_id, field_name)	Query by field
ACTIVITY_LOG	(tenant_id, site_id, created_at DESC)	Activity feed
USER_SITE_ACCESS	(user_id, site_id)	Auth check

Tenant Isolation (3 layers)

```
1. Every pipeline/support/extraction table carries tenant_id + site_id
2. JWT token holds { tenantId, siteId, role } — set at login
3. EF Core Global Query Filter auto-injects
   WHERE tenant_id = X AND site_id = Y  on EVERY query
```

```
modelBuilder.Entity<File>()
   .HasQueryFilter(f => f.TenantId == _currentUser.TenantId
                     && f.SiteId   == _currentUser.SiteId);
```

Even if a dev forgets a WHERE clause, the filter protects the data. Customer A can never see Customer B.

Key Business Rule — "Any file fails → transaction fails"

```
On any file status change:
  1. Update the 4 counters on its Transaction
  2. Recompute Transaction.state:
        failed_count > 0                              → Failed
        else uploaded_count + processing_count > 0    → Processing
        else                                           → Completed
  3. Update last_updated_at
  4. Write an ActivityLog entry
```

Failed-Files + Remediation Feature

The dashboard shows a Failed Files panel with columns: File Name | Error | 3-dots menu. Clicking the 3-dots menu reveals two actions:

- View Remediation — looked up from ErrorCatalog by error_code (turns ERR_BAD_SCHEMA into a human fix message).
- Download Logs — built from that file's FileStepHistory rows (full step-by-step trace).

Uses 3 tables together: Files (name) + FileStepHistory (error & steps) + ErrorCatalog (the fix).

What's shown	Source table	Column
File Name	Files	file_name
Error code	FileStepHistory	error_code
Remediation message	ErrorCatalog	remediation_msg (by error_code)
Download Logs	FileStepHistory	all step rows for that file

Confidence — how it is determined

PdfPig reads the real text layer (no built-in score), so confidence is derived from how cleanly we matched the value:

How value was found	Confidence
Clean regex / keyword match	0.95
Found via fallback guess	0.60
Not found	0.0

"Confidence reflects how reliable the extraction method was. If we later use OCR for scanned docs, we would store the OCR engine's real score instead."

Key Design Decisions

- UUID PKs → globally unique
- VARCHAR status → add new statuses without migrations
- Pre-aggregated counters → dashboard under 3–5s
- Denormalized tenant_id/site_id on Files → no JOIN on 1M rows
- doc_type_id on FileStepHistory → fast error panel without extra JOIN
- EAV (ExtractedFields) → unlimited doc types, zero migrations
- One row per step → queryable, indexable history
- Append-only ActivityLog → tamper-evident audit
- EF Core Migrations → version-controlled schema

Hand-off Notes for DT-2 (API Design)

With the schema locked, DT-2 should define endpoints around these core flows:

- Auth/login → issue JWT carrying { tenantId, siteId, role }.
- GET load-reporting list → paginated Transactions with the 4 counters + state + last_updated.
- GET transaction drill-down → list of Files in a transaction.
- GET file drill-down → FileStepHistory rows for a file.
- GET failed-files for a transaction → file name + error_code.
- GET remediation for a file → joins ErrorCatalog by error_code.
- GET file logs → downloadable log built from FileStepHistory.
- GET extracted fields for a file → rows from ExtractedFields.
- GET activity log → paginated, filtered by tenant/site.

All endpoints must respect tenant isolation (filter by tenant_id + site_id from the JWT) and target the 3–5s response budget using the indexes above.

— END OF DT-1 (FINAL) —

DT-2 — Technical API Specification (Detailed)

Document Processing Analytics Platform — refined design decision document. This revision commits to a single implementation choice for every open decision and expands endpoint coverage so that **every** functional requirement (FR-1 to FR-5) and non-functional requirement (NFR-1 to NFR-5) is satisfied.

Chosen technology stack (committed):

- **Backend:** ASP.NET Core Web API (controller-based, .NET 8).
- **ORM / Data access:** Entity Framework Core (parameterized LINQ → SQL, migration-based schema).
- **Database:** PostgreSQL (relational, satisfies the relational-DB constraint).
- **Auth:** JWT Bearer tokens validated by ASP.NET Core authentication middleware.
- **Docs:** Swagger / OpenAPI auto-generated from controllers.

---

1. Global Standards

Base URL & versioning: All endpoints are served under `/api/v1`. The version is in the path so breaking changes can ship as `/api/v2` without disrupting existing clients.

Authentication header: Every endpoint except `POST /auth/login` and `GET /health` requires `Authorization: Bearer <jwt>`. (NFR-3)

Date format: All timestamps are ISO-8601 UTC strings: `YYYY-MM-DDTHH:mm:ssZ` (e.g. `2023-10-25T14:30:00Z`).

Identifiers: All IDs are UUID (v4) strings, mapped to PostgreSQL `uuid` and .NET `Guid`.

Decision — Consistent response envelope (NFR-5)

Committed choice: every response uses one of two shapes. A single resource returns `{ data, error }`; a list returns `{ data, meta, error }`. On success `error` is `null`; on failure `data` is `null`. This single contract lets the frontend write one generic HTTP handler.

Standard list wrapper:

```
{
  "data": [ /* array of items */ ],
  "meta": {
    "total_count": 1250,
    "page": 1,
    "page_size": 50,
    "total_pages": 25
  },
  "error": null
}
```

Standard error envelope:

```
{
  "data": null,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "pageSize must be between 1 and 100",
    "details": { "field": "pageSize", "value": 10000 }
  }
}
```

Decision — Pagination strategy

Committed choice: offset-based pagination using `page` + `pageSize`. **Why this over cursor-based:** the UI needs jump-to-page and total-page counts (FR-1.4, FR-2.1, FR-4.4), which offset pagination supports natively. Cursor

pagination is faster on very deep pages but cannot show "Page 12 of 25". With proper indexes (DT-1) and a hard `pageSize` cap of 100, offset pagination meets the NFR-1 target of <1s for pages up to 50.

Decision — Filtering & sorting

Committed choice: query-string parameters for both (never request bodies on GET). Sorting uses `sortBy` (whitelisted column name) and `sortDir` (`asc`/`desc`). The `sortBy` value is validated against an allow-list to prevent SQL injection and is translated to an EF Core `OrderBy` expression — no string concatenation (NFR-3).

HTTP status codes used

Code	Meaning	When
200	OK	Successful GET / action
400	Bad Request	Validation failure (bad date, pageSize>100, bad enum)
401	Unauthorized	Missing / invalid / expired JWT
403	Forbidden	Authenticated but resource belongs to another tenant/site
404	Not Found	Resource ID does not exist at all
500	Server Error	Unhandled exception (logged, generic message returned)

---

2. Authentication & Identity

POST /api/v1/auth/login — entry point. Verifies the password hash against the USERS table, joins USER_SITE_ACCESS with SITES to discover which sites the user may view, and issues a JWT containing `userId`, `tenantId`, and `role`.

Request body:

```
{ "email": "user@org.com", "password": "secure_password" }
```

Success response (200):

```
{
  "data": {
    "token": "eyJhbGci...",
    "user": { "id": "uuid", "email": "...", "role": "Viewer" },
    "sites": [
      { "site_id": "uuid-1", "site_name": "Mumbai Plant" },
      { "site_id": "uuid-2", "site_name": "Delhi Warehouse" }
    ]
  },
  "error": null
}
```

GET /api/v1/auth/me — returns the current user profile and authorized sites from the token. Lets the SPA rehydrate session state after a page refresh without re-login. (Supports FR-5.1)

GET /api/v1/sites — returns the list of sites the authenticated user can access. Populates the global SiteSelector dropdown (FR-5.1). Returns 401 if unauthenticated.

---

3. Dashboard Endpoints (FR-1)

GET /api/v1/dashboard/summary — status counters (FR-1.1).

Logic: aggregate the TRANSACTIONS table for the selected site. `queued = SUM(total_files) - SUM(uploaded+processing+failed+completed)`; `failed = SUM(failed_count)`; and so on. Always filtered by `tenant_id` + `site_id`.

```
{
  "data": {
    "counters": { "queued":150, "in_progress":45,
                  "completed":8500, "failed":24 },
    "last_updated": "2023-10-25T14:30:00Z"
  },
  "error": null
}
```

GET /api/v1/dashboard/throughput?range=7d — throughput chart (FR-1.2).

Params: `range` = `24h` | `7d` | `30d` (validated enum). Logic: group `FILES` by `completed_at` bucket (hour for 24h, day otherwise) and count.

```
{
  "data": {
    "unit": "day",
    "series": [
      { "label":"2023-10-21", "value":450 },
      { "label":"2023-10-22", "value":510 }
    ]
  },
  "error": null
}
```

GET /api/v1/dashboard/status-distribution — status breakdown chart (FR-1.3).

Logic: count files grouped by current status for the selected site; feeds the pie/bar chart.

```
{
  "data": {
    "series": [
      { "label":"Completed", "value":8500 },
      { "label":"Failed", "value":24 },
      { "label":"In Progress", "value":45 },
      { "label":"Queued", "value":150 }
    ]
  },
  "error": null
}
```

GET /api/v1/dashboard/recent-failures — recent failures table (FR-1.4).

Paginated & sortable list of the most recent failed files: file name, failed step, error message, failed-at timestamp. Uses the standard list wrapper and accepts `page`, `pageSize`, `sortBy`, `sortDir`.

---

4. Batch Explorer (FR-2)

GET /api/v1/batches — paginated batch list (FR-2.1, FR-2.2, FR-2.3).

Query params:

- `page` : int (default 1)
- `pageSize` : int (default 20, max 100)
- `status` : enum all | in_progress | completed | failed (FR-2.2)
- `from`, `to` : ISO date range on submission time (FR-2.2)
- `source` : source-system filter (FR-2.2)
- `search` : full / partial Batch ID (FR-2.3)
- `sortBy`, `sortDir` : sorting

Logic: filters `TRANSACTIONS`; **must** include `WHERE tenant_id = X AND site_id = Y`. Columns returned: Batch ID, status, file count, submitted time, completed time, source system.

```
{
  "data": [
    { "transaction_id":"TID-9921", "status":"Failed",
      "source":"S3_Bucket_Alpha",
      "file_stats": { "total":100, "done":95, "error":5 },
      "times": { "started":"...", "updated":"..." } }
  ],
  "meta": { "total_count":120, "page":1,
            "page_size":20, "total_pages":6 },
  "error": null
}
```

GET /api/v1/batches/{batchId} — batch detail summary (FR-2.4): status, per-status file counts, start/end time.

GET /api/v1/batches/{batchId}/files — paginated files inside a batch (FR-2.4): file name, current status, current step, last-updated time. **Solves N+1** by loading files in one query instead of per-batch lookups (DT-4).

GET /api/v1/files/{fileId}/details — file step history (FR-2.5 & FR-3.3). Joins `FILES + FILE_STEP_HISTORY + ERROR_CATALOG`.

```
{
  "data": {
    "file_info": { "id":"uuid", "name":"invoice_oct.pdf",
      "current_status":"Failed", "current_step":"Extraction" },
    "history": [
      { "step":"Upload", "status":"Success",
        "ts":"2023-10-21T10:00:00Z" },
      { "step":"Extraction", "status":"Failed",
        "ts":"2023-10-21T10:05:00Z",
        "error": { "code":"ERR_OCR_40",
          "message":"Text unreadable on page 2",
          "suggested_fix":"Re-scan the document at 300DPI." } }
    ]
  },
  "error": null
}
```

---

5. Error Analysis (FR-3)

GET /api/v1/errors/top-frequencies — top 10 errors (FR-3.1, FR-3.3).

Logic: scan `FILE_STEP_HISTORY` where `status = 'Failed'`, group by `error_code`, order by count desc, limit 10; joins `ERROR_CATALOG` for the remediation text.

```
{
  "data": [
    { "code":"INVALID_DATE", "count":450,
      "remediation":"Update date format to YYYY-MM-DD" },
    { "code":"SCHEMA_MISMATCH", "count":210,
      "remediation":"Verify CSV column headers" }
  ],
  "error": null
}
```

GET /api/v1/errors/trend?range=30d — error trend chart (FR-3.2): number of failures per day over the range. Same `series` shape as throughput.

GET /api/v1/errors — filtered, paginated error list (FR-3.4). Filters:

- `from`, `to` : date range
- `step` : processing step where the error occurred
- `source` : source system
- `page`, `pageSize`, `sortBy`, `sortDir`

GET /api/v1/errors/export — CSV export of the filtered error list (FR-3.5). Accepts the same filter params as `/errors` and responds with `Content-Type: text/csv` and a `Content-Disposition: attachment` header. The export is still tenant/site-scoped by the middleware.

---

6. Activity Log (FR-4)

GET /api/v1/activity-log — paginated, chronological audit trail (FR-4.1–FR-4.4).

Each entry includes timestamp, event type, related entity (batch ID or file name), old state, new state, and the actor that triggered it. Filterable by `eventType`, `entity`, and `from`/`to` date range; paginated (never loads all rows).

```
{
  "data": [
    { "ts":"2023-10-25T14:30:00Z",
      "event_type":"FILE_STATE_CHANGED",
      "entity":"invoice_oct.pdf",
      "old_state":"In Progress", "new_state":"Failed",
      "actor":"system" }
  ],
  "meta": { "total_count":980, "page":1,
            "page_size":50, "total_pages":20 },
  "error": null
}
```

---

7. Reliability (NFR-4)

GET /api/v1/health — unauthenticated health check that verifies database connectivity (opens a lightweight EF Core query). Returns 200 with `{ "status":"healthy", "db":"connected" }` or 503 when the DB is unreachable. Used by load balancers and uptime monitors.

---

8. Security & Business Rules (NFR-3)

Rule 1 — JWT + Tenant/Site middleware

A middleware runs before every controller. It (1) validates the incoming JWT, (2) extracts `tenantId` and `role`, (3) reads the requested `site_id`, and (4) attaches them to the request context. EF Core **global query filters** then automatically append `WHERE tenant_id = @ctx.TenantId` to every query — developers cannot forget it. This is how FR-5.2 / FR-5.3 isolation is guaranteed at the data layer.

Rule 2 — Input validation

All inputs are validated before hitting the DB. `pageSize` is capped at 100 — a request for `pageSize=10000` returns 400 Bad Request. Date params must parse as ISO-8601; `status`/`range` must match the allowed enums; IDs must be valid UUIDs.

Rule 3 — 404 vs 403

For `/api/v1/files/123`: if the ID does not exist anywhere, return **404 Not Found**; if it exists but belongs to another tenant/site, return **403 Forbidden** — never reveal that another tenant's resource exists.

Rule 4 — Parameterized queries only

All data access goes through EF Core LINQ, which emits parameterized SQL. No string concatenation of user input into SQL anywhere (NFR-3).

Rule 5 — Role-based access (S-3 ready)

The `role` claim supports future RBAC: `Admin` may switch across all tenants, `Viewer` is locked to assigned sites. Enforced with ASP.NET Core `[Authorize(Roles=...)]` attributes.

Summary of Committed Decisions — DT-2

Decision	Committed Choice	Why
Response format	Envelope { data, meta, error }	One consistent contract for the whole UI (NFR-5)
Pagination	Offset-based (page/pageSize, max 100)	Supports jump-to-page & total counts; meets <1s target
Filtering & sorting	Whitelisted query params	Cacheable, injection-safe, RESTful
Auth	JWT Bearer + middleware	Tenant/site isolation enforced centrally
Tenant isolation	EF Core global query filters	Cannot be forgotten by developers (FR-5.2/5.3)
Errors	404 vs 403 + standard error body	No info leak across tenants

Endpoint coverage: FR-1 (summary, throughput, status-distribution, recent-failures), FR-2 (batches, batch detail, batch files, file details), FR-3 (top-frequencies, trend, filtered list, CSV export), FR-4 (activity-log), FR-5 (login, me, sites + middleware), plus NFR-4 health check.

DT-3 — Frontend Architecture (Detailed)

Document Processing Analytics Platform — refined design decision document. Each decision now commits to a single implementation choice and is expanded with concrete structure so the build can begin directly.

Chosen framework (committed): Angular 17+ using standalone components and the modern Signals reactivity model. This pairs cleanly with the ASP.NET Core API in DT-2 and matches the team's existing skill set.

---

1. Pages / Routes

Committed choice: Nested / Parameterized Routes (Option B). The site context and resource IDs live in the URL, so any view is bookmarkable and deep-linkable, and the browser Back button works correctly. Each feature area is **lazy-loaded** so the initial bundle stays small (helps the NFR-1 3-second dashboard load).

Why not the others: flat routes (A) can't deep-link to a specific batch; pure state routing (C) breaks Back/refresh and bookmarking.

Route table:

Route	Page	Requirement
/login	Authentication	FR-5.1
/site/:siteId/dashboard	Dashboard overview	FR-1
/site/:siteId/batches	Batch list	FR-2.1–2.3
/site/:siteId/batches/:batchId	Batch detail	FR-2.4
/site/:siteId/batches/:batchId/files/:fileId	File step history	FR-2.5
/site/:siteId/errors	Error analysis	FR-3
/site/:siteId/activity-log	Audit trail	FR-4

Route guards: an `authGuard` blocks all routes except `/login` when no token is present; a `siteAccessGuard` verifies the `:siteId` in the URL is one the user is authorized for (mirrors the server's FR-5.3 check, fail-fast on the client).

---

2. Reusable Components

Committed choice: Atomic Design. Components are layered atoms → molecules → organisms so a "Failed" badge looks identical on all five pages and shared logic is written once.

Atoms (small, presentational)

- **StatusBadge** — takes a status string, returns a coloured pill (Failed=red, Completed=green, In Progress=blue, Queued=grey).
- **StatCard** — a single number + title tile used on the dashboard counters.
- **AppButton** — button with built-in loading-spinner state (NFR-2).

Molecules / Organisms (structural)

- **DataTable** — generic table handling pagination, sorting and empty/loading states, reused for batches, files, recent failures, errors and the activity log.
- **SiteSelector** — the tenant/site dropdown in the top nav, present on every page (FR-5.1).
- **RefreshTimer** — shows "Refreshing in X s" and triggers the dashboard reload (FR-1.5).

- **ChartCard** — wrapper around the charting library for throughput, distribution and error-trend charts (FR-1.2, FR-1.3, FR-3.2).
- **FilterBar** — reusable status / date-range / source filters shared by Batches, Errors and Activity Log.

---

3. State Management

Committed choice: Angular Signals inside injectable Services (Option C). Services own the data (a signal per slice); components read it via `computed()` and the template updates automatically when a signal changes. **Why not NgRx (B):** for a mid-size app it is over-engineering — 5x more boilerplate. **Why not bare BehaviorSubjects (A):** Signals offer the same benefit with simpler syntax and fine-grained change detection.

Core services:

- **AuthService** — holds the JWT + current user signal; handles login/logout and session rehydration via `/auth/me`.
- **SiteContextService** — holds the `selectedSiteId` signal (see section 4).
- **DashboardService, BatchService, ErrorService, ActivityLogService** — one per feature; each exposes `data`, `loading` and `error` signals so every page can render loading indicators and friendly error messages (NFR-2).

```
// SiteContextService (illustrative)
selectedSiteId = signal<string | null>(null);
setSite(id: string) { this.selectedSiteId.set(id); }
```

---

4. Global Tenant / Site Selection

Committed choice: Global Service (Option B) — a SiteContextService holding the current selection, kept in sync with the URL. The `:siteId` route param is the source of truth; the SiteSelector navigates to a new URL on change, and a small effect mirrors that param into the service signal. Every API call reads `selectedSiteId`, satisfying FR-5.2 (all data scoped to the selected site).

Why this over pure URL-only (A) or LocalStorage (C): the URL keeps views bookmarkable while the central service keeps every chart/table in sync without prop-drilling IDs through components. LocalStorage alone makes cross-component sync fragile.

Implementation — HTTP interceptor: an Angular interceptor automatically attaches the auth token and the current `site_id` to every outgoing request, so individual components never manage it.

```
// authSiteInterceptor (illustrative)
const token = auth.token();
const siteId = siteCtx.selectedSiteId();
req = req.clone({
  setHeaders: { Authorization: `Bearer ${token}` },
  params: req.params.set('site_id', siteId ?? '')
});
```

Switching site changes the URL → the param effect updates the signal → dependent `computed()`/effects re-fetch → all pages reload their data (FR-5.2).

---

5. Dashboard Auto-Refresh

Committed choice: RxJS polling (Option B). A `timer(0, 30000)` stream pulses every 30 seconds; on each pulse the dashboard refreshes the summary, throughput, distribution and recent-failures data (FR-1.5). **Why over setInterval (A):** RxJS composes cleanly with cancellation and is declarative. **Why over WebSocket (C):** sockets are a stretch goal (S-1) and add backend complexity not needed for v1.

Cleanup & correctness:

- Use `takeUntilDestroyed()` so the timer stops when the user leaves the dashboard — prevents memory leaks and stray requests.
- The interval is configurable (FR-1.5 says “configurable interval”).
- Pause polling when the browser tab is hidden (`document.visibilityState`) to save bandwidth.
- The RefreshTimer atom shows the countdown and lets the user trigger an immediate manual refresh.

```
// Dashboard polling (illustrative)
timer(0, this.intervalMs)
  .pipe(
    switchMap(() => this.dashboard.loadSummary()),
    takeUntilDestroyed(this.destroyRef)
  )
  .subscribe();
```

---

6. Cross-Cutting Concerns

- **Error handling (NFR-2/NFR-4):** a global HTTP error interceptor maps the API’s `{ error }` envelope to a toast + inline message and offers a retry — never a blank screen.
- **Loading states (NFR-2):** every service exposes a `loading` signal driving skeletons / spinners.
- **Responsive layout (NFR-2):** CSS grid / flex tested from 1024px to 1920px; a persistent sidebar or top nav across all pages.
- **Separation of concerns (NFR-5):** components render, services hold state + API calls, interceptors handle auth/site/errors.
- **Dark mode (S-2 ready):** theme via CSS variables, preference persisted to LocalStorage.

Summary of Committed Decisions — DT-3

Problem	Committed Choice	Why
Routing	Nested / parameterized routes	Bookmarkable, deep-linkable, Back-button safe
Components	Atomic design	Consistent UI & write-once logic across pages
State	Signals inside services	Fastest dev time, fine-grained reactivity, no boilerplate
Tenant sync	Global SiteContextService + URL	All data scoped to site; components stay in sync (FR-5.2)
Auto-refresh	RxJS polling (timer 30s)	Declarative, cancellable, no socket complexity (FR-1.5)
Auth/site plumbing	HTTP interceptor	Token + site_id attached automatically